

Computing the Code Distance of Quantum Error-Correcting Codes

State-of-the-Art Review

review draft

Generated 2026-06-09 from a /survey knowledge base (25 references).

Scope

This report assesses how the **minimum distance** d of a quantum error-correcting code is computed in practice. It is written for someone deciding which tool or method to use, or scoping research in the area. The review is organized *by technical approach*: each method family is described with its mechanism, current best results, and trade-offs. Code **construction** and **decoding** are out of scope except where they bear on distance computation.

1. What and Why

A quantum error-correcting code stores k logical qubits in n physical qubits via a **stabilizer group** S of commuting Pauli operators. Its **distance** d is the minimum weight of a Pauli operator that commutes with every stabilizer but is not itself a stabilizer — the lightest non-trivial **logical** operator, $d = \min \text{wt}(N(S) \setminus S)$ [1], [2]. A distance- d code detects $d - 1$ errors and corrects $\lfloor d/2 \rfloor$, so d sets the fault-tolerance budget directly. Unlike n and k , which are read off the construction, d must be **computed** — and that is the problem this report surveys.

Why it is hard, and why that matters now: distance computation is a minimum-weight codeword search over a coset, the same combinatorial object that makes the classical minimum-distance problem **NP-hard** [3], [4]. The quantum version inherits this and more — it is NP-hard to compute and even to approximate [5], [6], while **degeneracy** makes the related optimal-decoding problem #P-complete [7]. As hardware groups move to high-rate qLDPC codes with hundreds to thousands of qubits, certifying their distance has become a live bottleneck rather than a textbook exercise.

Working in the symplectic representation, each Pauli maps to a length- $2n$ binary vector $(x|z)$, stabilizers form a check matrix H , and d is the minimum symplectic weight of a logical operator — a vector e with $He = 0$ but $Le \neq 0$ [2], [8]. What distinguishes the field is the resulting **split by guarantee** (Figure 1): **exact** methods certify d by squeezing matching lower and upper bounds together but cost exponentially; **heuristic** methods only ever exhibit a light logical operator, returning an upper bound with quantified confidence. Every approach below sits on one side of this line.

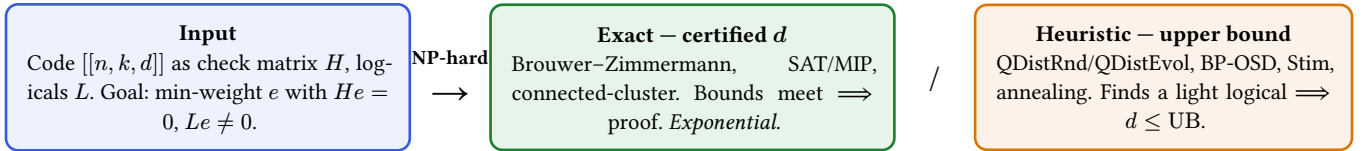


Figure 1: One problem, two solution families with fundamentally different guarantees.

2. Technical Approaches

The lineage runs from classical hardness foundations to quantum-specific tooling (Figure 2). Six method families are in active use; the first three certify d , the last three return upper bounds. A cross-approach summary closes the section.

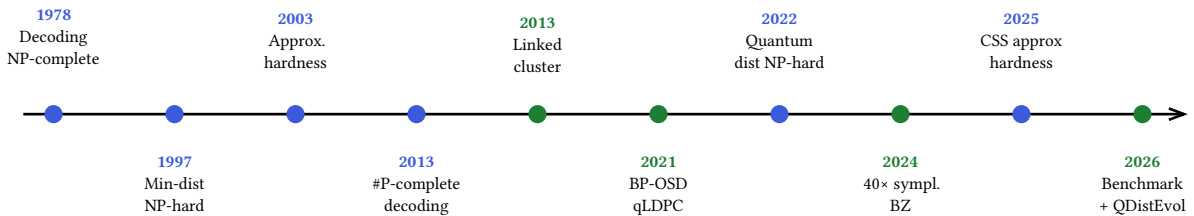


Figure 2: Field landscape. Blue = complexity/hardness results; green = algorithms and tooling.

2.1. Exact enumeration: Brouwer–Zimmernann

What it is. Build several (partial) information sets by repeated reduced row-echelon form on the generator matrix over disjoint column blocks, then enumerate weight- t row combinations of each systematic generator for increasing t . Each information set yields a lower bound on the weight of not-yet-seen codewords; when the summed lower bound reaches the lightest codeword found, the search is **certified complete** [2], [9]. The stabilizer version computes the symplectic weight of the normalizer matrix [8].

State of the art. The fastest implementation is the symplectic Brouwer–Zimmermann of Hernando–Quintana-Orti–Grassl, **up to $40 \times$ faster than Magma** and turning “days into seconds” on shared-memory multicore [8]; its classical predecessor is ACM Algorithm 994 [9]. Magma’s proprietary BZ remains the strongest exact baseline on small non-CSS codes (98.7% solved within 8 h in the 2026 benchmark) [2].

Strengths

- Certifies d with a proof, not just a bound [2].
- Even/doubly-even weight jumps and Gray-code enumeration accelerate termination [2].
- Mature, openly available high-performance implementations [8], [9].

Limitations

- Exponential cost; times out on large lifted-product / bivariate-bicycle codes [2].
- The strongest baseline (Magma) is proprietary and cannot separate X/Z distance [2], [10].

2.2. Constraint solvers: SAT and MIP

What it is. Encode “minimum-weight e with $He = 0$ and $Le \neq 0$ ” directly for an off-the-shelf solver. MaxSAT uses soft clauses to penalize each error bit and Tseitin-encoded XOR clauses for the parity constraints; mixed-integer programming minimizes Hamming weight subject to mod-2 constraints expressed with integer slack variables [2].

State of the art. In the 2026 benchmark, MIP (Gurobi) is the best **exact** method for bivariate-bicycle and quantum-Tanner codes — precisely the high-rate families where cluster search and BZ time out — and found the lowest distance on 4 of 7 of the hardest bivariate-bicycle **circuits** [2]. The 2-block symplectic encoding outperforms the 3-block one for Gurobi [2].

Strengths

- Certified, and inherits decades of solver engineering [2].
- Solves structured high-rate codes that other exact methods cannot finish [2].

Limitations

- Exponential in the number of integer variables [2].
- Orders of magnitude slower than BZ on small codes (Gurobi $\sim 6.7 \times 10^3$ s vs Magma ~ 31 s) [2].
- On the hardest families it completes only the smallest members within 8 h [2].

2.3. Connected-cluster search

What it is. The support of a minimum-weight codeword forms a single connected cluster on the Tanner-derived graph, so only growing **connected** clusters need be enumerated by breadth-first search — exponentially fewer candidates than generic weight- w errors [11], [12].

State of the art. It is the fastest **exact** method on hyperbolic-surface and colour CSS codes — clearing all 21 surface codes in 1.0 s versus 2.2×10^5 s for Magma — and its cost exponent is **linear** in the relative distance [2], [11].

Strengths

- Cost linear in relative distance; exploits LDPC sparsity [11], [12].
- Beats other deterministic methods at small relative distance and random-window at high rate [11], [12].
- Certified, and the fastest exact route on surface/colour codes [2].

Limitations

- Cluster count explodes with Tanner-graph degree; times out on bivariate-bicycle codes [2].
- The C implementation yields no partial lower bound if terminated early [2], [11].

2.4. Probabilistic and evolutionary sampling: QDistRnd / QDistEvol

What it is. A random column permutation followed by RREF surfaces a low-weight logical operator; iterating tightens the upper bound. QDistRnd is the random-window form; QDistEvol replaces random restarts with a genetic search over permutations driven by a continuous (minimum-weight, average-weight) fitness [13], [14].

State of the art. QDistRnd is the de-facto-standard GAP package, with a quantified failure probability $P_{\text{fail}} < e^{-\langle n \rangle}$ from repeat counts [13]. The evolutionary QDistEvol is the clear winner on high-rate qLDPC in the 2026 benchmark: lowest distance on 13/20 bivariate-bicycle codes (vs 8/20 for QDistRnd and BP-OSD) and 19/19 lifted-product codes; on a 756-qubit code its per-trial success is ~ 50 –60% against $\sim 2\%$ for plain QDistRnd [2], [14].

Strengths

- Scales to thousands of qubits where exact search is hopeless [2], [13].
- Quantified failure probability from repeated low-weight finds [13].
- QDistEvol gives the best accuracy and throughput on qLDPC codes [2].

Limitations

- Upper bound only — never proves no lighter logical exists [2], [13].
- Highly sensitive to representation/permutation (a $[[48, 5, 10]]$ code went 0% $\rightarrow$ 100% after DEM permutation + stabilizer mixing) [2].
- QDistEvol needs structure: only $\sim 27\%$ success on unstructured code-table codes [2].

2.5. Decoder- and circuit-based search: BP-OSD and Stim

What it is. Pick a random non-trivial logical, then decode the syndrome that is zero on checks but anti-commutes with it; belief-propagation + ordered-statistics decoding (BP-OSD) returns a low-weight logical operator. Stim instead searches the **detector error model** of a full syndrome-extraction circuit for an undetectable logical error, giving the **circuit-level** distance [2], [15], [16].

State of the art. BP-OSD was established to estimate distances of generalized-bicycle qLDPC codes [15]. Stim analyzes a distance-100 surface-code circuit (20k qubits, 8M gates) in ~ 15 s, and its graphlike-error search is **exact** whenever every error flips at most two detectors [2], [16].

Strengths

- The only practical route to **circuit-level** distance, what governs real experiments [2], [16].
- Stim’s graphlike search is exact for string-like errors (e.g. surface codes) [2].
- BP-OSD is competitive on very sparse, large CSS codes [2].

Limitations

- Upper bound only outside the graphlike-exact regime [2].
- BP-OSD never clearly beat plain random-information-set sampling on any benchmark dataset [2].
- BP convergence is hampered by length-4 loops in qLDPC Tanner graphs [11], [15].

2.6. Optimization metaheuristics: annealing / QUBO and quantum ISD

What it is. Recast the minimum-distance problem as a QUBO (with only logarithmic multiplicative overhead in variables for the mod-2 arithmetic) and solve it on a quantum annealer, hybrid solver, or by simulated annealing [10]; or use quantum-walk-accelerated information-set decoding to search for low-weight codewords [17].

State of the art. The QUBO/annealing approach is “competitive, on par with the best classical QUBO solvers” but still lags the best **deterministic** distance algorithms [10]; quantum ISD lowers the asymptotic exponent for finding low-weight codewords of random linear codes [17]. This family is the least mature — promising but not yet a production tool.

Strengths

- Only logarithmic variable overhead in the QUBO encoding [10].
- May ride future quantum-hardware scaling [10]; quantum ISD has a proven asymptotic speedup [17].

Limitations

- Upper bound only, with no optimality certificate [10].
- Currently lags the best deterministic methods; the advantage may vanish at scale [10].
- Quantum ISD presumes fault-tolerant hardware that does not yet exist [17].

2.7. Shared infrastructure

Cutting across the approaches: Stim supplies the fast stabilizer simulation and detector-error-model machinery many circuit-level methods build on [16]; the Munich Quantum Toolkit bundles code-analysis utilities [18]; and best-known-distance references come from Grassl’s `codetables.de` and quantum-code databases [19], which are how any computed bound is checked against the literature.

2.8. At-a-glance comparison

Approach	Scalability	Verifiability / cost	Maturity	Best-fit use case
Brouwer–Zimmermann	Small–medium n [8]	Certified; exponential [2]	Mature [8]	Small non-CSS codes, code tables [2]
SAT / MIP	Moderate, structured [2]	Certified; exponential [2]	Maturing [2]	High-rate qLDPC where cluster/BZ time out [2]

Approach	Scalability	Verifiability / cost	Maturity	Best-fit use case
Connected-cluster	Linear in d , degree-sensitive [11]	Certified [2]	Mature [2]	Sparse LDPC, surface/colour codes [2]
QDistRnd / QDistEvol	Thousands of qubits [2]	Upper bound; P_{fail} [13]	Mature [13]	High-rate qLDPC (QDistEvol wins) [2]
BP-OSD / Stim	Very large / circuits [16]	Upper bound (Stim exact if graphlike) [2]	Mature [16]	Circuit-level distance, very sparse checks [2]
Annealing / ISD	Emerging [10]	Upper bound; no certificate [10]	Experimental [17]	Research; future hardware [10]

3. Open Problems

The open problems descend from one root — NP-hardness — into the practical bottlenecks (Figure 3).

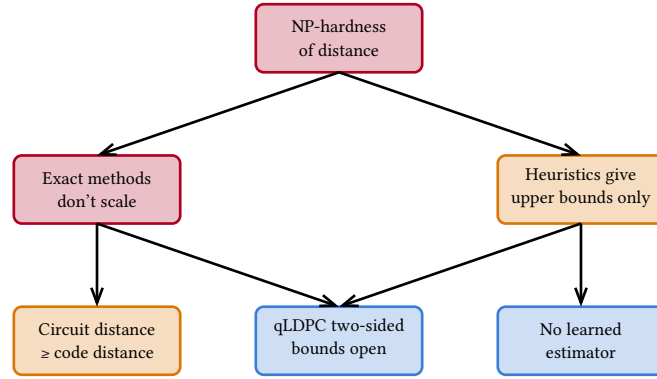


Figure 3: Problem dependencies. Red = critical, orange = high, blue = medium urgency.

No.	Problem	Why it matters	Who could solve it	Urgency
1	Exact distance at scale [2], [8]	No certified method finishes within 8 h for large lifted-product / bivariate-bicycle codes — exactly those being built for hardware.	Better SAT/MIP encodings; faster symplectic BZ; partial-LB cluster search (Webster, Grassl, Higgott).	Critical
2	Upper-bound-only heuristics [2], [13]	Every scalable method can silently overstate d ; a wrong d undermines the code's fault-tolerance guarantee.	Confidence-bounded sampling beyond QDistRnd's P_{fail} ; certified hybrids (Pryadko, UCL).	Critical
3	Circuit vs code distance [2], [16]	Detector-error models are $\sim 100 \times$ larger than check matrices; circuit distance is what governs experiments.	Scalable DEM-aware search (Google Quantum AI / Stim; UCL codeDistance).	High
4	Square-root approximation gap [5], [6]	No hardness known for additive gap τn^ϵ with $1/2 < \epsilon \leq 1$; a conjectured " $\sqrt{n}$ barrier" limits reductions.	Complexity theory (Grigorescu, Guruswami, Kapshikar groups).	High
5	Two-sided qLDPC bounds [12], [15]	Numerical methods give upper bounds, expansion arguments give lower bounds; closing the gap is still case-by-case.	Coding theorists on bivariate-bicycle / APM-LDPC families [20], [21].	Medium
6	No learned distance estimator [14]	ML is used for decoding and code discovery but not for estimating d — a possible fast-screening gap.	ML-for-QEC groups; open opportunity.	Medium

Bottom line

No single method dominates. For **certified** distance, pick by structure: Brouwer–Zimmermann (Magma or the $40 \times$ symplectic implementation) on small non-CSS codes [8], connected-cluster on surface/colour codes, and MIP on high-rate families where the others time out [2]. For codes too large to certify — the qLDPC regime hardware actually targets — the evolutionary QDistEvol is the current best **upper-bound** tool, with Stim for circuit-level distance [2], [16]. The standing caveat is that every scalable method returns an upper bound, so a reported d on a large code is a working estimate, not a proof [2], [13].

Sources: 25-reference knowledge base built by /survey on 2026-06-09. Three pre-arXiv IEEE classics (Berlekamp 1978, Vardy 1997, Dumer 2003) are cited from metadata; all others are grounded in rendered full text. The central comparative source is the Webster–Jacob–Higgott 2026 benchmark [2].

References

- [1] D. Gottesman, “Stabilizer Codes and Quantum Error Correction,” *arXiv: Quantum Physics*, 1997.
- [2] M. A. Webster, A. Jacob, and O. Higgott, “Distance-Finding Algorithms for Quantum Codes and Circuits,” 2026.
- [3] A. Vardy, “The intractability of computing the minimum distance of a code,” *IEEE Trans. Inf. Theory*, vol. 43, pp. 1757–1766, 1997.
- [4] I. Dumer, D. Micciancio, and M. Sudan, “Hardness of approximating the minimum distance of a linear code,” *IEEE Transactions on Information Theory*, vol. 49, no. 1, pp. 22–37, Jan. 2003, doi: [10.1109/tit.2002.806118](https://doi.org/10.1109/tit.2002.806118).
- [5] U. S. Kapshikar and S. Kundu, “On the Hardness of the Minimum Distance Problem of Quantum Codes,” *IEEE Transactions on Information Theory*, vol. 69, pp. 6293–6302, 2022.
- [6] E. Grigorescu, V. Jha, and E. Samperton, “On the hardness of approximating minimum distances of quantum codes,” *ArXiv*, 2025.
- [7] P. Iyer and D. Poulin, “Hardness of Decoding Quantum Stabilizer Codes,” *IEEE Transactions on Information Theory*, vol. 61, pp. 5209–5223, 2013.
- [8] F. Hernando, G. Quintana-Ortí, and M. Grassl, “Fast Algorithms and Implementations for Computing the Minimum Distance of Quantum Codes,” *ACM Transactions on Quantum Computing*, vol. 7, pp. 1–19, 2024.
- [9] F. Hernando, F. D. Igual, and G. Quintana-Ortí, “Algorithm 994,” *ACM Transactions on Mathematical Software (TOMS)*, vol. 45, pp. 1–28, 2016.
- [10] R. Ismail, A. Kakkar, and A. Dymarsky, “A quantum annealing approach to the minimum distance problem of quantum codes,” 2024.
- [11] A. Kovalev, I. Dumer, and L. Pryadko, “Linked-cluster technique for finding the distance of a quantum LDPC code,” *2013 Information Theory and Applications Workshop (ITA)*, pp. 1–6, 2013.
- [12] I. Dumer, A. Kovalev, and L. Pryadko, “Numerical techniques for finding the distances of quantum codes,” *2014 IEEE International Symposium on Information Theory*, pp. 1086–1090, 2014.
- [13] L. Pryadko, V. A. Shabashov, and V. K. Kozin, “QDistRnd: A GAP package for computing the distance of quantum error-correcting codes,” *J. Open Source Softw.*, vol. 7, p. 4120, 2022.
- [14] M. A. Webster and D. E. Browne, “Engineering Quantum Error Correction Codes Using Evolutionary Algorithms,” *IEEE Transactions on Quantum Engineering*, vol. 6, pp. 1–14, 2024.
- [15] P. Panteleev and G. Kalachev, “Degenerate Quantum LDPC Codes With Good Finite Length Performance,” *Quantum*, vol. 5, p. 585, 2019.
- [16] C. Gidney, “Stim: a fast stabilizer circuit simulator,” *Quantum*, vol. 5, p. 497, 2021.
- [17] G. Kachigar and J. Tillich, “Quantum Information Set Decoding Algorithms,” *IACR Cryptol. ePrint Arch.*, vol. 2017, p. 213, 2017.
- [18] R. Wille *et al.*, “The MQT Handbook : A Summary of Design Automation Tools and Software for Quantum Computing,” *2024 IEEE International Conference on Quantum Software (QSW)*, pp. 1–8, 2024.
- [19] N. Aydin, P. Liu, and B. Yoshino, “A Database of Quantum Codes,” *ArXiv*, 2021.
- [20] K. Kasai, “Heuristic Search for Minimum-Distance Upper-Bound Witnesses in Quantum APM-LDPC Codes,” 2026.
- [21] N. Delfosse, Z. Li, and S. Thomassé, “A Note on the Minimum Distance of Quantum LDPC Codes,” pp. 239–250, 2014.